



L'Extension ARM

Projet Genie Logiciel - gl16

ENSIMAG 2025/2026

AIT MOHAMED Younes (aitmohay)
BEAURAIN Tom (beurato)
BEN ISMAIL Wajd (benismaw)
FURLANI DA SILVA SOARES Manuela (furlanim)
MARIA ENGLER PINTO Carlos (mariaenc)

Table des matières

1	Introduction	3
2	Analyse bibliographique	4
2.1	Génération de code dans les compilateurs	4
2.2	Rôle des machines abstraites	4
2.3	Architecture ARM : principes généraux	4
2.4	Contraintes de la génération de code pour ARM	5
2.5	Syntaxe générale de l'assembleur ARM	5
2.6	Jeu de registres ARM	5
2.7	Instructions ARM effectivement utilisées dans l'extension	6
2.8	Instructions ARM natives utilisées	6
2.9	Instructions non natives implémentées via un runtime	7
2.10	Justification des choix d'implémentation	8
2.11	Accès mémoire et gestion de la pile	8
2.12	Instructions ARM pour les flottants	9
2.13	Appels système et environnement Linux	9
2.14	Lien avec la génération de code Deca	9
2.15	Double back-end et portabilité	10
2.16	Apports de l'analyse bibliographique au projet	10
3	Analyse et conception	10
3.1	Objectifs et périmètre de l'extension ARM	10
3.2	Architecture générale du compilateur et double back-end	11
3.3	Conception du back-end ARM et rôle de <code>ARMProgram</code>	11
3.4	Absence d'environnement d'exécution et conception du runtime	12
3.5	Implémentation du modulo et contraintes matérielles	12
3.6	Architecture logicielle détaillée du back-end ARM	12
3.7	Hierarchie des instructions ARM	13
3.8	Modélisation des opérandes ARM	13
3.9	Gestion explicite des registres entiers et flottants	14
3.10	Rôle central de la classe <code>ARMProgram</code>	14
3.11	Injection conditionnelle du runtime ARM	14
3.12	Bilan des choix de conception du back-end ARM	14
4	Choix d'algorithmes	15
4.1	Stratégie générale de génération de code	15
4.2	Algorithmes de traduction des expressions arithmétiques	15
4.3	Gestion algorithmique du modulo sur ARM	16
4.4	Algorithmes de gestion des registres	16
4.5	Algorithmes pour les instructions d'affichage	16
4.6	Algorithmes liés à la pile et aux appels de fonctions	17
4.7	Gestion algorithmique des erreurs et robustesse	17
4.8	Synthèse des choix algorithmiques	17

5	Méthode de validation	17
5.1	Validation automatisée par les scripts de génération ARM	18
5.2	Analyse des erreurs sur les tests invalides	18
5.3	Validation manuelle par compilation et exécution ciblées	19
5.4	Débogage par lecture directe du code assembleur généré	19
5.4.1	Inspection du fichier <code>.ass</code>	19
5.4.2	Analyse des registres et de la pile	20
5.4.3	Lien entre comportement observé et code assembleur	20
5.4.4	Limites de cette approche de débogage	20
5.5	Comportements non déterministes et erreurs de type <i>Bus error</i>	21
5.6	Validation incrémentale et correction par commits ciblés	21
5.7	Validation du runtime ARM et des entrées/sorties	21
5.8	Synthèse de la méthode de validation	22
6	Validation de l'implémentation	22
6.1	Couverture fonctionnelle effective	22
6.2	Cas partiellement couverts et comportements incorrects	23
6.3	Erreurs critiques et instabilités d'exécution	23
6.4	Limites structurelles du compilateur ARM	24
6.5	Bilan global de l'implémentation	24

1 Introduction

Le projet de Génie Logiciel (GL) s'inscrit dans une démarche pédagogique visant à concevoir progressivement un compilateur complet pour un langage impératif simplifié, nommé Deca. Ce compilateur repose sur une architecture classique en plusieurs phases : analyse lexicale, analyse syntaxique, analyse contextuelle, puis génération de code. Chaque étape vise à illustrer concrètement les concepts fondamentaux de la compilation, depuis la reconnaissance du langage source jusqu'à la production d'un code exécutable.

Dans sa version initiale, le compilateur Deca génère du code pour une machine abstraite pédagogique. Ce choix permet de se concentrer sur la sémantique du langage et la structure du compilateur, tout en masquant volontairement les contraintes matérielles d'une architecture réelle. Toutefois, cette abstraction limite la compréhension des problématiques concrètes liées à la génération de code bas niveau, telles que la gestion explicite des registres, de la pile ou des conventions d'appel.

Nous avons donc proposé et implémenté une extension du compilateur Deca visant à produire du code assembleur pour une architecture réelle : ARM. Cette extension constitue un changement d'échelle significatif, en confrontant directement la génération de code à des contraintes matérielles strictes et à l'absence de primitives de haut niveau. Elle permet ainsi de mettre en évidence l'écart existant entre une génération de code conceptuelle et une génération de code réellement exécutable sur une architecture matérielle.

L'objectif de cette extension n'est pas de produire un compilateur ARM industriel ou optimisé, mais de concevoir un back-end fonctionnel, cohérent et pédagogiquement pertinent. Il s'agit notamment de :

- adapter la génération de code existante à une architecture *load/store*,
- gérer explicitement les registres entiers et flottants,
- implémenter un runtime ARM minimal pour les entrées/sorties et certaines opérations absentes du jeu d'instructions,
- assurer la compatibilité avec l'architecture existante du compilateur via un double back-end.

Cette extension met en lumière des problématiques centrales de la compilation bas niveau, telles que la discipline des registres, la gestion de la pile, les conventions d'appel et le débogage par analyse du code assembleur généré. Elle illustre également la difficulté de garantir un comportement déterministe lorsque le compilateur cible une architecture réelle sans environnement d'exécution complet.

Ce document présente de manière détaillée :

- les fondements théoriques de la génération de code et de l'architecture ARM,
- les choix de conception retenus pour le back-end ARM,
- les algorithmes utilisés pour traduire les constructions du langage Deca,
- la méthodologie de validation adoptée, incluant le débogage par lecture du code assembleur,
- une analyse critique de l'implémentation et de ses limitations.

L'ensemble de ce travail vise à démontrer qu'un compilateur pédagogique peut être étendu de manière cohérente vers une architecture réelle, tout en mettant en évidence les difficultés techniques et conceptuelles inhérentes à la génération de code bas niveau.

2 Analyse bibliographique

2.1 Génération de code dans les compilateurs

La génération de code constitue l'une des étapes finales d'un compilateur. Après l'analyse lexicale, syntaxique et contextuelle, le compilateur dispose d'une représentation intermédiaire du programme, généralement sous la forme d'un arbre de syntaxe abstraite (AST). Le rôle du back-end est alors de traduire cette représentation intermédiaire en un code exécutable pour une architecture cible donnée.

Dans la littérature classique sur les compilateurs, notamment dans *Compilers : Principles, Techniques, and Tools* de Aho *et al.*, la génération de code est souvent présentée comme un compromis entre simplicité, portabilité et performance. Afin de limiter la complexité, de nombreux compilateurs s'appuient sur une machine abstraite ou une représentation intermédiaire avant de produire du code spécifique à une architecture réelle.

Cette approche permet notamment :

- de factoriser la logique de compilation,
- de faciliter l'ajout de nouvelles cibles matérielles,
- de séparer les problématiques de sémantique du langage de celles liées à l'architecture.

2.2 Rôle des machines abstraites

Une machine abstraite est une architecture virtuelle simplifiée, conçue pour servir d'étape intermédiaire entre le langage source et le matériel réel. Elle dispose généralement :

- d'un jeu d'instructions réduit et régulier,
- d'un nombre abstrait de registres,
- d'une sémantique simple et bien définie.

Dans un cadre pédagogique, l'utilisation d'une machine abstraite permet de concentrer l'effort sur la sémantique du langage, sans être immédiatement confronté aux contraintes matérielles d'un processeur réel.

Cependant, une machine abstraite ne reflète pas toujours fidèlement les limitations d'une architecture concrète. Certaines opérations peuvent y être directement disponibles (par exemple le modulo), alors qu'elles ne le sont pas nécessairement sur un processeur réel. Le passage d'une machine abstraite à une architecture matérielle nécessite donc une adaptation non triviale.

2.3 Architecture ARM : principes généraux

L'architecture ARM est une architecture de type RISC (*Reduced Instruction Set Computer*). Elle se caractérise par :

- un jeu d'instructions simple,
- une forte utilisation des registres,
- des instructions généralement exécutées en un nombre de cycles réduit.

ARM est aujourd'hui l'une des architectures les plus répandues au monde. Elle est omniprésente dans :

- les systèmes embarqués,
- les smartphones et tablettes,

- de plus en plus dans les serveurs et centres de calcul.

Cette large adoption a conduit à une documentation abondante, notamment à travers les manuels d'architecture ARM, les ABI (*Application Binary Interface*) et les spécifications des appels système sous Linux.

2.4 Contraintes de la génération de code pour ARM

Contrairement à une machine abstraite, l'architecture ARM impose des contraintes strictes concernant :

- l'utilisation des registres,
- les conventions d'appel de fonctions,
- la préservation des registres lors des appels (`b1`),
- l'absence de certaines instructions de haut niveau.

Par exemple, ARM ne fournit pas d'instruction native pour le calcul du modulo. Celui-ci doit être reconstruit à partir d'opérations élémentaires telles que la division, la multiplication et la soustraction. Cette limitation, bien documentée dans les manuels ARM, illustre la nécessité de choix algorithmiques explicites lors de la génération de code.

De plus, en l'absence d'un environnement d'exécution complet, le compilateur doit fournir un *runtime* minimal, permettant de gérer les entrées/sorties ainsi que certaines opérations arithmétiques complexes.

2.5 Syntaxe générale de l'assembleur ARM

L'assembleur ARM adopte une syntaxe relativement lisible et régulière, caractéristique des architectures de type RISC. Les instructions sont généralement constituées d'un mnémotechnique suivi d'opérandes explicites, le plus souvent des registres ou des adresses mémoire.

Une instruction ARM typique s'écrit sous la forme :

```
instruction destination, source1, source2
```

Par exemple :

```
add r0, r1, r2
```

effectue l'addition des registres `r1` et `r2`, et place le résultat dans `r0`.

La syntaxe ARM se distingue par :

- une séparation claire entre registres et mémoire,
- l'absence d'opérations mémoire-mémoire,
- l'utilisation fréquente d'instructions conditionnelles.

Dans le cadre du projet Deca, cette syntaxe a été exploitée directement lors de la génération du code assembleur, via la classe `ARMProgram`, sans passer par une représentation intermédiaire supplémentaire.

2.6 Jeu de registres ARM

L'architecture ARM repose sur un ensemble de registres généraux notés `r0` à `r15`. Dans le cadre de l'ABI ARM utilisée sous Linux :

- **r0** à **r3** servent principalement au passage des paramètres et des valeurs de retour,
- **r4** à **r11** sont des registres généraux,
- **r13** correspond au pointeur de pile (**sp**),
- **r14** est le registre de lien (**lr**),
- **r15** est le compteur ordinal (**pc**).

Dans l'extension ARM du compilateur Deca, les registres **r0** et **r1** sont largement utilisés pour :

- l'évaluation des expressions,
- le passage des paramètres aux routines du runtime,
- le stockage temporaire des résultats intermédiaires.

En complément, l'architecture ARM propose des registres flottants (**s0**, **s1**, etc.), utilisés pour les calculs en virgule flottante. Leur intégration dans le projet a nécessité une gestion spécifique et une distinction claire entre registres entiers et flottants.

2.7 Instructions ARM effectivement utilisées dans l'extension

L'extension ARM du compilateur Deca ne repose pas sur l'intégralité du jeu d'instructions ARM, mais sur un sous-ensemble ciblé, suffisant pour couvrir les constructions du langage Deca. Ces instructions peuvent être classées en deux grandes catégories :

- les instructions ARM natives, directement supportées par l'architecture,
- les instructions de haut niveau implémentées sous forme de runtime, absentes du jeu d'instructions ARM.

Cette distinction est fondamentale pour comprendre les choix d'implémentation et les limitations observées lors de la génération de code.

2.8 Instructions ARM natives utilisées

Les instructions ARM natives correspondent à des opérations directement supportées par le processeur. Elles sont traduites quasi directement depuis la sémantique du langage Deca.

Instructions arithmétiques entières Les opérations arithmétiques de base sont implémentées à l'aide des instructions suivantes :

- **ARMAdd** : addition entière (**add**),
- **ARMSub** : soustraction entière (**sub**),
- **ARMMul** : multiplication entière (**mul**),
- **ARMReverseSub** : soustraction inversée (**rsb**).

Ces instructions permettent d'implémenter directement les opérateurs **+**, **-**, ***** du langage Deca. Elles illustrent bien la philosophie RISC d'ARM, où chaque instruction correspond à une opération élémentaire.

Comparaisons et branchements Les comparaisons reposent sur l'instruction :

- **ARMCmp** : comparaison de deux registres (**cmp**).

Cette instruction met à jour les indicateurs de condition du processeur, utilisés ensuite par les branchements conditionnels :

- **ARMBeq** : branchement si égalité,

- ARMBne : branchement si différence,
- ARMBlt, ARMBle, ARMBgt, ARMBge : comparaisons strictes et larges.

Ces instructions constituent la base de l'implémentation des structures de contrôle (if, else, while) et des opérateurs relationnels du langage.

Chargement, stockage et mouvement de données L'architecture ARM étant de type *load/store*, les accès mémoire sont réalisés exclusivement via :

- ARMLdr, ARMLdrMem, ARMLdrFloat : chargement depuis la mémoire,
- ARMStr, ARMStrb : stockage en mémoire,
- ARMMov : déplacement de valeurs entre registres.

Ces instructions sont utilisées pour la gestion des variables, des affectations et des paramètres.

Pile et appels de fonctions La gestion explicite de la pile repose sur :

- ARMPush,
- ARMPop.

Les appels de fonctions utilisent :

- ARMB1 : branchement avec sauvegarde de l'adresse de retour.

Ces instructions sont essentielles pour respecter les conventions d'appel ARM et garantir la non-interférence entre le code utilisateur et le runtime.

Appels système Les interactions avec le système Linux sont réalisées via :

- ARMSvc : appel système (`svc 0`).

Cette instruction est utilisée pour l'écriture sur la sortie standard et la terminaison du programme.

2.9 Instructions non natives implémentées via un runtime

Certaines opérations du langage Deca ne disposent pas d'instructions ARM natives équivalentes. Elles ont donc été implémentées sous forme de routines de runtime injectées dans le programme généré.

Division entière ARM ne fournit pas toujours d'instruction de division entière simple et portable selon les variantes et les contextes d'exécution. Pour garantir un comportement cohérent, une instruction abstraite :

- ARMDiv

a été implémentée sous forme de routine de runtime.

Cette routine réalise une division par soustractions successives, avec une vérification explicite de la division par zéro. Ce choix, bien que peu performant, garantit la correction sémantique et la portabilité dans le cadre du projet pédagogique.

Entrées standard Les opérations de lecture :

- ARMReadInt,
- ARMReadFloat

ne correspondent pas à des instructions ARM, mais à des abstractions de haut niveau.

Elles reposent sur des appels système Linux combinés à des routines de conversion, permettant de lire une valeur depuis l'entrée standard et de la stocker dans un registre ou en mémoire.

Affichage L'affichage constitue un autre exemple d'opération absente du jeu d'instructions ARM. Les instructions suivantes ont donc été implémentées comme des appels à un runtime :

- `ARMPrintInt`,
- `ARMPrintFloat`,
- `ARMPrintString`,
- `ARMPrintln`.

Ces routines prennent en charge :

- la conversion des valeurs en représentation textuelle,
- l'écriture sur la sortie standard,
- la gestion correcte des registres et de la pile.

Le choix de séparer l'affichage du reste de la génération de code permet de conserver un back-end simple, tout en centralisant les opérations complexes dans le runtime.

2.10 Justification des choix d'implémentation

Le recours à un runtime pour certaines opérations répond à plusieurs contraintes :

- l'absence d'instructions ARM natives équivalentes,
- la nécessité de garantir un comportement déterministe,
- la volonté de conserver une génération de code lisible et maintenable.

Ces choix illustrent parfaitement l'écart entre une machine abstraite pédagogique et une architecture réelle. Ils mettent en évidence que certaines opérations considérées comme élémentaires à haut niveau nécessitent une implémentation explicite et rigoureuse au niveau assembleur.

Cette analyse des instructions utilisées constitue un socle théorique essentiel pour comprendre les difficultés rencontrées lors de l'implémentation et de la validation de l'extension ARM.

2.11 Accès mémoire et gestion de la pile

ARM adopte une architecture *load/store*, ce qui signifie que seules les instructions de chargement et de stockage accèdent à la mémoire. Les instructions principales utilisées dans le projet sont :

- `ldr` pour charger une valeur depuis la mémoire,
- `str` pour stocker une valeur en mémoire.

Par exemple :

```
ldr r0, [r1]
str r0, [r1]
```

La pile est manipulée explicitement à l'aide des instructions :

- `push {...}`,

— `pop {...}`.

Ces instructions sont essentielles pour préserver les registres lors des appels aux routines du runtime ARM, notamment pour les fonctions d’affichage (`print`, `println`).

2.12 Instructions ARM pour les flottants

Les calculs flottants utilisent des instructions spécifiques de l’extension VFP de l’architecture ARM. Parmi celles utilisées dans le projet :

- `vadd.f32`, `vsub.f32`, `vmul.f32`,
- `vcvt.f32.s32` pour la conversion int vers float,
- `vcvt.s32.f32` pour la conversion inverse,
- `vcmp.f32` pour les comparaisons flottantes.

Après une comparaison flottante, il est nécessaire de transférer les indicateurs vers le registre d’état général à l’aide de l’instruction :

```
vmrs APSR_nzcv, fpscr
```

Cette étape est indispensable pour permettre l’utilisation d’instructions de branchement conditionnel standard (`beq`, `bne`).

2.13 Appels système et environnement Linux

L’exécution des programmes ARM générés repose sur les appels système Linux, réalisés via l’instruction :

```
svc 0
```

Avant l’appel, les registres sont positionnés conformément à l’ABI Linux ARM :

- `r7` contient le numéro de l’appel système,
- `r0`, `r1`, `r2` contiennent les paramètres.

Cette mécanique est utilisée dans le runtime ARM minimal pour implémenter :

- l’écriture sur la sortie standard,
- la gestion des retours à la fin du programme.

2.14 Lien avec la génération de code Deca

L’étude de la syntaxe et des instructions ARM a directement influencé la conception de l’extension ARM du compilateur Deca. Chaque contrainte syntaxique ou matérielle identifiée lors de l’analyse bibliographique s’est traduite par un choix concret dans la génération de code :

- séparation stricte entre calcul et accès mémoire,
- gestion explicite des conversions de types,
- implémentation manuelle des opérations absentes du jeu d’instructions.

Cette analyse approfondie de l’assembleur ARM constitue ainsi un socle théorique essentiel pour comprendre les choix de conception et les limitations observées dans l’implémentation du back-end ARM.

2.15 Double back-end et portabilité

La littérature sur les compilateurs met en avant l'intérêt des architectures à double back-end, dans lesquelles un même front-end peut produire du code pour plusieurs cibles. Cette approche est utilisée par de nombreux compilateurs industriels, notamment via des représentations intermédiaires communes, comme LLVM IR.

Dans le cadre de ce projet, l'existence d'un back-end pour une machine abstraite et d'un second back-end pour l'architecture ARM illustre concrètement ce principe de portabilité. Elle met en évidence les différences entre :

- une génération de code conceptuelle et simplifiée,
- une génération de code réaliste, soumise à des contraintes matérielles.

2.16 Apports de l'analyse bibliographique au projet

Cette analyse bibliographique a permis :

- d'identifier les rôles respectifs du front-end et du back-end,
- de comprendre l'intérêt pédagogique d'une machine abstraite,
- d'anticiper les difficultés liées à l'architecture ARM,
- de justifier les choix de conception retenus pour l'extension.

Elle a notamment mis en évidence que la génération de code pour une architecture réelle nécessite une gestion rigoureuse des registres, une connaissance précise des conventions d'appel et une implémentation explicite des fonctionnalités absentes du jeu d'instructions.

3 Analyse et conception

Cette section décrit de manière détaillée l'analyse technique et les choix de conception retenus pour l'extension ARM du compilateur Deca. Elle s'appuie sur les contraintes concrètes rencontrées lors du développement et sur l'architecture existante du projet. L'objectif n'était pas uniquement de produire du code assembleur fonctionnel, mais de concevoir une extension cohérente, maintenable et compatible avec la structure pédagogique du compilateur.

3.1 Objectifs et périmètre de l'extension ARM

L'extension ARM a pour objectif principal de permettre au compilateur Deca de générer du code assembleur exécutable sur une architecture ARM réelle, en complément du back-end existant basé sur une machine abstraite. Cette extension s'inscrit dans une logique pédagogique visant à confronter une génération de code conceptuelle à une génération de code soumise à de véritables contraintes matérielles.

Le périmètre fonctionnel de l'extension couvre :

- la génération de code ARM pour les expressions arithmétiques,
- la gestion des variables et des affectations,
- l'implémentation des instructions d'affichage (`print` et `println`),
- le support d'opérations non natives comme le modulo,
- la production d'un programme ARM complet, exécutable via un outil d'émulation.

En revanche, l'extension ne vise pas à fournir un environnement d'exécution complet comparable à une bibliothèque standard. Les fonctionnalités sont volontairement limitées afin de conserver une complexité maîtrisée.

3.2 Architecture générale du compilateur et double back-end

Le compilateur Deca repose sur une architecture classique en plusieurs phases : analyse lexicale, analyse syntaxique, analyse contextuelle, puis génération de code. L'extension ARM intervient exclusivement dans la phase de génération de code, sans modifier les phases amont.

Le projet dispose déjà d'un back-end fonctionnel produisant du code pour une machine abstraite. L'ajout du support ARM a conduit à une architecture à double back-end, dans laquelle un même arbre de syntaxe abstraite est exploité par deux générateurs de code distincts.

Concrètement, chaque nœud pertinent de l'arbre syntaxique (expressions, instructions, déclarations) implémente désormais une méthode de génération de code ARM, en parallèle de la génération existante. Par exemple, les classes représentant les instructions d'affichage définissent une méthode dédiée :

```
@Override
protected void codeGenInstARM(DecacCompiler compiler) {
    AbstractExpr arg = getArguments().getList().get(0);
    arg.codeGenExprARM(compiler);

    if (arg.getType().isInt()) {
        compiler.getARMProgram().addInstruction(new ARMPrintInt());
    } else if (arg.getType().isFloat()) {
        compiler.getARMProgram().addInstruction(new ARMPrintFloat());
    } else if (arg.getType().isString()) {
        compiler.getARMProgram().addInstruction(new ARMPrintString());
    }

    compiler.getARMProgram().addInstruction(new ARMPrintln());
}
```

Ce choix permet de conserver une correspondance directe entre les constructions du langage Deca et le code ARM généré, tout en évitant toute duplication du front-end.

3.3 Conception du back-end ARM et rôle de ARMProgram

La génération de code ARM repose sur une classe centrale, `ARMProgram`, chargée de construire progressivement le programme assembleur final. Cette classe joue un rôle analogue à celui du générateur de code pour la machine abstraite, mais en tenant compte des spécificités d'ARM.

`ARMProgram` est responsable :

- de l'ajout des instructions assembleur ARM,
- de la gestion des sections (`.text`, `.data`),

- de l'injection du runtime ARM minimal,
- de la génération du point d'entrée du programme.

Le code assembleur produit est ensuite compilé et exécuté à l'aide d'un script fourni dans le projet, par exemple :

```
./arm.sh examples/test_print.ass
```

Ce script automatise la compilation avec `gcc` et l'exécution via un environnement ARM, ce qui permet de tester rapidement le code généré.

3.4 Absence d'environnement d'exécution et conception du runtime

L'une des principales difficultés de l'extension ARM réside dans l'absence d'un environnement d'exécution complet. Contrairement à la machine abstraite, ARM ne fournit pas nativement de primitives de haut niveau pour l'affichage ou certaines opérations arithmétiques.

Pour pallier cette limitation, un runtime ARM minimal a été conçu et injecté directement dans le programme généré. Ce runtime contient notamment :

- des routines d'affichage pour les entiers, flottants et chaînes de caractères,
- la gestion des appels système Linux via l'instruction `svc`,
- la sauvegarde et la restauration des registres conformément aux conventions ARM.

Par exemple, l'affichage d'un entier repose sur une routine assembleur appelée depuis le code généré par `ARMPrintInt`. Ce découplage entre le code utilisateur et le runtime permet de simplifier la génération de code tout en garantissant une exécution correcte.

3.5 Implémentation du modulo et contraintes matérielles

Un cas emblématique des différences entre machine abstraite et architecture réelle est l'opération de modulo. Alors que la machine abstraite propose une instruction dédiée, ARM ne fournit pas d'instruction native pour le modulo.

L'implémentation du modulo repose donc sur une décomposition explicite :

$$a \bmod b = a - (a/b) \times b$$

Cette opération est générée à partir d'instructions ARM élémentaires (division, multiplication, soustraction), ce qui a nécessité une attention particulière à la gestion des registres temporaires et à la préservation des valeurs intermédiaires.

Ce choix illustre parfaitement l'un des objectifs pédagogiques de l'extension : montrer que certaines opérations considérées comme simples à haut niveau nécessitent une implémentation explicite lorsqu'on cible une architecture réelle.

3.6 Architecture logicielle détaillée du back-end ARM

Le back-end ARM du compilateur Deca a été conçu selon une architecture orientée objet, en cohérence avec l'architecture existante du compilateur et en particulier avec le back-end basé sur la machine abstraite. L'objectif principal était de conserver une

structure homogène entre les deux cibles, tout en intégrant les contraintes spécifiques de l'architecture ARM.

Cette architecture repose sur une modélisation explicite des différents éléments de l'assembleur ARM à l'aide de classes Java dédiées. Chaque concept fondamental du langage assembleur est représenté par une abstraction logicielle claire, ce qui permet de structurer la génération de code et de limiter les erreurs de conception.

Le back-end ARM s'articule principalement autour des éléments suivants :

- une hiérarchie d'instructions ARM,
- une hiérarchie d'opérandes (registres, immédiats, adresses mémoire),
- une gestion explicite des registres entiers et flottants,
- une classe centrale chargée de construire le programme assembleur final.

Cette structuration permet une génération de code typée, lisible et maintenable, au prix d'une complexité logicielle plus importante que celle de la machine abstraite.

3.7 Hiérarchie des instructions ARM

Les instructions ARM sont modélisées par une hiérarchie de classes centrée autour de la classe abstraite `ARMInstruction`. Cette classe représente une instruction assembleur générique, indépendante de son nombre d'opérandes.

La hiérarchie distingue plusieurs grandes catégories d'instructions :

- `ARMNullaryInstruction` pour les instructions sans opérandes,
- `ARMUnaryInstruction` pour les instructions à un opérande,
- `ARMBinaryInstruction` pour les instructions à deux opérandes,
- `ARMBranchInstruction` pour les instructions de branchement.

Des spécialisations supplémentaires permettent de représenter précisément les contraintes de l'architecture ARM, notamment en distinguant :

- les instructions manipulant des adresses mémoire,
- celles manipulant des valeurs immédiates,
- celles opérant exclusivement sur des registres.

Cette hiérarchie empêche la génération d'instructions invalides et garantit que les opérandes utilisés sont compatibles avec le type d'instruction émis.

3.8 Modélisation des opérandes ARM

Les opérandes ARM sont représentés par une hiérarchie dédiée, centrée sur la classe abstraite `ARMOperand`. Cette abstraction permet de modéliser précisément la diversité des opérandes disponibles dans l'architecture ARM.

On distingue notamment :

- les registres généraux via `ARMRegister` et `ARMGPRRegister`,
- les constantes immédiates (`ARMImmediateInteger`, `ARMImmediateFloat`, `ARMImmediateString`),
- les adresses mémoire indexées (`ARMRegisterOffset`),
- les opérandes symboliques (`ARMLabelOperand`).

Cette modélisation est essentielle pour respecter le modèle *load/store* de l'architecture ARM, dans lequel seules les instructions de chargement et de stockage accèdent à la mémoire.

3.9 Gestion explicite des registres entiers et flottants

La génération de code ARM impose une gestion explicite et rigoureuse des registres, contrairement à la machine abstraite où cette gestion est largement simplifiée. Le projet distingue clairement :

- les registres entiers (`r0` à `r15`),
- les registres flottants (`s0`, `s1`, etc.).

Les registres entiers sont modélisés par la classe `ARMGPRegister`, tandis que les registres flottants sont manipulés séparément afin d'éviter toute confusion de type. Cette séparation stricte s'est révélée indispensable pour corriger plusieurs erreurs observées lors de l'exécution de programmes combinant entiers et flottants.

L'absence d'allocation globale de registres impose une discipline stricte dans l'utilisation des registres temporaires, notamment lors de l'évaluation d'expressions complexes et des appels au runtime.

3.10 Rôle central de la classe `ARMProgram`

La classe `ARMProgram` constitue le cœur du back-end ARM. Elle est responsable de la construction progressive du programme assembleur final, en orchestrant :

- la section `.data`,
- le préambule `.text` et le point d'entrée `main`,
- la liste ordonnée des instructions ARM générées,
- l'injection conditionnelle du runtime ARM.

Contrairement à une simple concaténation de chaînes de caractères, `ARMProgram` maintient une structure interne composée de listes distinctes pour les données, les instructions et les portions de code assembleur brut. Cette organisation garantit un contrôle précis sur l'ordre de génération et facilite le débogage du code produit.

3.11 Injection conditionnelle du runtime ARM

Une particularité importante de la conception du back-end ARM réside dans l'injection conditionnelle du runtime. Les routines de runtime ne sont ajoutées au programme que si elles sont effectivement utilisées.

Par exemple :

- l'utilisation de `ARMPrintInt` déclenche l'injection de la routine `__print_int`,
- `ARMPrintFloat` entraîne l'injection de `__print_float`,
- `ARMDiv` injecte la routine `__div`,
- `ARMReadInt` et `ARMReadFloat` injectent les routines de lecture correspondantes.

Ce mécanisme repose sur des indicateurs internes à `ARMProgram`, garantissant que chaque routine de runtime n'est injectée qu'une seule fois. Ce choix permet de produire un code assembleur plus compact et plus lisible, tout en évitant toute duplication inutile.

3.12 Bilan des choix de conception du back-end ARM

Les choix de conception retenus pour le back-end ARM traduisent un compromis entre réalisme architectural, simplicité d'implémentation et objectifs pédagogiques. La modélisation explicite de l'assembleur ARM, la séparation claire entre instructions, opérandes

et runtime, ainsi que l'introduction d'un mécanisme d'injection conditionnelle constituent des fondations solides pour l'extension.

Bien que cette approche introduise une complexité plus importante que celle de la machine abstraite, elle reflète fidèlement les contraintes d'une architecture réelle et met en évidence les difficultés inhérentes à la génération de code bas niveau. Elle constitue ainsi un support pertinent pour analyser, valider et critiquer l'extension ARM du compilateur Deca.

4 Choix d'algorithmes

Cette section présente les algorithmes retenus pour la génération de code ARM dans le compilateur Deca. Contrairement à la machine abstraite, la cible ARM impose des contraintes fortes sur le jeu d'instructions, la gestion des registres et les conventions d'appel. Les choix algorithmiques effectués visent à garantir la correction fonctionnelle du code généré, tout en conservant une implémentation lisible, progressive et adaptée au cadre pédagogique du projet.

4.1 Stratégie générale de génération de code

La génération de code ARM repose sur une stratégie descendante, pilotée par l'arbre de syntaxe abstraite. Chaque nœud pertinent de l'arbre est responsable de la génération du code correspondant à sa sémantique, via une méthode dédiée de type `codeGenExprARM` ou `codeGenInstARM`.

Ce choix algorithmique permet :

- une correspondance directe entre les constructions du langage Deca et le code ARM généré,
- une génération incrémentale du programme assembleur,
- une localisation précise des erreurs de génération.

La génération de code suit un modèle impératif simple : les instructions ARM sont ajoutées séquentiellement à une instance de `ARMProgram`, sans phase d'optimisation globale. Ce choix est volontaire afin de privilégier la clarté et la traçabilité du code généré.

4.2 Algorithmes de traduction des expressions arithmétiques

La traduction des expressions arithmétiques constitue une part essentielle du back-end ARM. Les expressions sont évaluées récursivement, chaque sous-expression générant du code qui place son résultat dans un registre.

L'algorithme retenu repose sur les principes suivants :

- évaluation récursive des sous-expressions,
- utilisation explicite de registres temporaires,
- respect strict de l'ordre d'évaluation.

Par exemple, pour une expression binaire de type `a + b`, le schéma algorithmique est le suivant :

1. générer le code ARM pour `a` et placer le résultat dans un registre,
2. sauvegarder ce registre si nécessaire,

3. générer le code ARM pour `b`,
4. appliquer l'instruction ARM correspondant à l'opération.

Cette approche garantit la correction du calcul, mais nécessite une gestion rigoureuse des registres afin d'éviter les écrasements de valeurs intermédiaires.

4.3 Gestion algorithmique du modulo sur ARM

Le calcul du modulo représente un cas particulier, car ARM ne fournit pas d'instruction native pour cette opération. Contrairement à la machine abstraite, où le modulo est directement disponible, il a fallu concevoir un algorithme explicite.

L'algorithme retenu repose sur l'identité mathématique :

$$a \bmod b = a - (a/b) \times b$$

La génération de code suit donc les étapes suivantes :

1. calcul de la division entière `a / b`,
2. multiplication du quotient par `b`,
3. soustraction du produit à `a`.

Chaque étape est traduite en instructions ARM élémentaires, ce qui implique l'utilisation de plusieurs registres temporaires. Une attention particulière a été portée à la préservation des opérandes initiaux, notamment lorsque ceux-ci sont eux-mêmes le résultat d'expressions complexes.

Ce choix algorithmique illustre clairement l'écart entre une génération de code abstraite et une génération de code pour une architecture réelle, où certaines opérations de haut niveau doivent être reconstruites manuellement.

4.4 Algorithmes de gestion des registres

La gestion des registres constitue l'un des points les plus sensibles de la génération de code ARM. Contrairement à la machine abstraite, le nombre de registres est limité et leur utilisation est soumise à des conventions strictes.

L'algorithme retenu repose sur une allocation simple et déterministe :

- utilisation prioritaire de registres temporaires pour les expressions,
- sauvegarde explicite des registres avant les appels de fonctions,
- restauration systématique après les appels.

Aucune stratégie avancée de type allocation globale ou graphe d'interférences n'a été mise en place. Ce choix est cohérent avec les objectifs pédagogiques du projet et permet de conserver un comportement prévisible du générateur de code.

Les appels à des routines du runtime ARM, comme les fonctions d'affichage, respectent les conventions d'appel ARM, notamment la sauvegarde des registres non volatils via des instructions de type `push` et `pop`.

4.5 Algorithmes pour les instructions d'affichage

Les instructions `print` et `println` sont implémentées via un algorithme en deux phases :

1. génération du code de l'expression à afficher,
2. appel d'une routine du runtime ARM correspondant au type de la valeur.

Le choix de la routine dépend du type statique de l'expression, déterminé lors de l'analyse contextuelle. Par exemple, une expression entière déclenche l'appel à `ARMPrintInt`, tandis qu'une expression flottante appelle `ARMPrintFloat`.

Ce découplage entre génération du calcul et génération de l'affichage permet :

- une simplification du code généré,
- une centralisation des opérations d'E/S dans le runtime,
- une meilleure maintenabilité du back-end ARM.

4.6 Algorithmes liés à la pile et aux appels de fonctions

Les appels aux routines du runtime ARM nécessitent une gestion correcte de la pile et des registres. L'algorithme retenu repose sur :

- la sauvegarde des registres utilisés avant l'appel (`push`),
- l'appel via l'instruction `bl`,
- la restauration des registres après le retour (`pop`).

Cette approche garantit la non-interférence entre le code utilisateur généré et le runtime ARM, au prix d'un surcoût en instructions. Ce compromis est acceptable dans le cadre du projet, où la correction prime sur les performances.

4.7 Gestion algorithmique des erreurs et robustesse

L'extension ARM ne dispose pas d'un mécanisme avancé de gestion des erreurs à l'exécution. Les algorithmes de génération ont donc été conçus pour minimiser les situations d'erreur, notamment en :

- évitant les divisions par zéro lorsque cela est détectable statiquement,
- générant du code simple et prévisible,
- testant systématiquement les nouvelles fonctionnalités sur des cas minimaux.

Les erreurs restantes se manifestent principalement lors de l'exécution du code ARM, ce qui a nécessité une phase de débogage approfondie, décrite dans les sections suivantes.

4.8 Synthèse des choix algorithmiques

Les algorithmes retenus pour la génération de code ARM privilégient la clarté, la robustesse et la conformité aux contraintes de l'architecture. Le refus volontaire d'optimisations complexes permet de mettre en évidence les mécanismes fondamentaux de la compilation vers une architecture réelle.

Ces choix algorithmiques constituent un compromis adapté au cadre pédagogique du projet et fournissent une base saine pour la validation et l'évaluation de l'extension ARM.

5 Méthode de validation

La validation de l'extension ARM du compilateur Deca a constitué une part essentielle du travail, en raison de la complexité inhérente à la génération de code bas niveau pour une

architecture réelle. Contrairement au back-end de la machine abstraite, la génération ARM expose directement les erreurs de gestion des registres, de la pile et du runtime, souvent sous la forme de comportements non déterministes ou de fautes d'exécution difficiles à diagnostiquer.

La méthode de validation adoptée repose sur une combinaison de tests automatisés fournis par le projet, de tests unitaires manuels, d'exécutions répétées via un environnement ARM, et d'une analyse fine des erreurs rencontrées.

5.1 Validation automatisée par les scripts de génération ARM

Le projet met à disposition un script spécifique dédié à la validation de la génération de code ARM :

```
./src/test/script/test-gencode-ARM.sh
```

Ce script exécute automatiquement un ensemble de tests couvrant la génération de code pour l'architecture ARM, incluant environ 60 fichiers Deca répartis entre cas valides et cas invalides.

Une exécution typique de ce script a produit le résultat suivant :

```
Total tests exécutés : 60
Tests REUSSIS       : 42
Tests ECHOUES       : 18
```

Les tests valides couvrent des cas variés : expressions arithmétiques complexes, structures de contrôle (`if`, `while`), gestion des booléens, conversions implicites, chaînes de caractères et stress tests de registres (par exemple `heavy_stack.deca` ou `reg_priority_test.deca`). Leur succès progressif a permis de valider une grande partie du back-end ARM.

Les échecs observés se répartissent principalement en deux catégories :

- des tests invalides pour lesquels une erreur à l'exécution était attendue,
- des tests révélant des problèmes structurels dans la gestion des registres ou du runtime.

5.2 Analyse des erreurs sur les tests invalides

Plusieurs tests invalides, notamment ceux liés à la division par zéro, ont mis en évidence une différence entre le comportement attendu par le script de test et le comportement réel du runtime ARM.

Par exemple, pour un test tel que `div_par_zero.deca`, l'exécution ARM produisait bien un message explicite :

```
Erreur : division par zero
```

Cependant, le script de validation considérait ce cas comme un échec, car l'erreur n'était pas propagée sous la forme attendue (arrêt brutal du programme). Ce décalage souligne une limitation actuelle du runtime ARM, qui signale certaines erreurs mais ne provoque pas systématiquement une interruption conforme aux attentes du cadre de test.

5.3 Validation manuelle par compilation et exécution ciblées

En complément des scripts automatisés, une validation manuelle a été menée par compilation et exécution répétées de fichiers Deca individuels. Cette approche s'est révélée indispensable pour diagnostiquer des erreurs fines liées à la génération ARM.

La chaîne de validation utilisée est la suivante :

```
decac -arm ./src/test/deca/codegen/valid/sans_objet/complex.deca  
./arm.sh ./src/test/deca/codegen/valid/sans_objet/complex.ass
```

Ces commandes ont été exécutées de nombreuses fois sur les mêmes fichiers, ce qui a permis d'observer des comportements instables particulièrement révélateurs de problèmes de gestion de registres.

5.4 Débogage par lecture directe du code assembleur généré

Contrairement au back-end basé sur la machine abstraite, le back-end ARM ne bénéficie pas d'outils de débogage de haut niveau intégrés. En pratique, la majorité des erreurs rencontrées lors de la génération de code ARM ne sont pas détectables à la compilation, mais uniquement à l'exécution, souvent sous la forme de résultats incohérents ou d'erreurs critiques (*Bus error, core dumped*).

Dans ce contexte, la lecture directe du code assembleur généré (fichier `.ass`) est devenue un outil central de débogage. Cette approche consiste à analyser manuellement le code ARM produit par le compilateur afin de comprendre précisément :

- quelles instructions sont émises,
- dans quel ordre elles sont exécutées,
- quels registres sont utilisés, modifiés ou écrasés,
- comment la pile est manipulée lors des appels au runtime.

Cette méthode, bien que coûteuse en temps, s'est révélée indispensable pour identifier des erreurs impossibles à diagnostiquer via les seuls messages d'exécution.

5.4.1 Inspection du fichier `.ass`

Après compilation d'un programme Deca avec l'option ARM, le compilateur produit un fichier assembleur intermédiaire de type `.ass`. Ce fichier contient :

- la section `.data`, incluant les chaînes, buffers et constantes du runtime,
- la section `.text`, contenant le code généré pour le programme utilisateur,
- les routines du runtime ARM injectées conditionnellement.

La lecture de ce fichier permet de vérifier directement si la génération de code correspond bien à la sémantique attendue du programme Deca. Par exemple, pour une expression arithmétique simple, il est possible de vérifier :

- si les chargements (`ldr`) sont correctement effectués,
- si les registres intermédiaires sont cohérents,
- si le résultat final est bien placé dans le registre attendu avant un affichage.

5.4.2 Analyse des registres et de la pile

Une part importante du débogage par lecture du code assembleur a consisté à suivre manuellement l'évolution des registres. Cette analyse est particulièrement critique dans les cas suivants :

- expressions imbriquées,
- appels successifs à des routines du runtime,
- mélanges d'opérations entières et flottantes.

La lecture du code `.ass` permet notamment d'identifier :

- des registres écrasés prématurément,
- des sauvegardes manquantes avant un `bl`,
- des restaurations incorrectes via `pop`,
- des incohérences entre registres entiers (`rX`) et flottants (`sX`).

Plusieurs erreurs de type *Bus error* ont ainsi pu être reliées directement à un désalignement de la pile ou à une restauration incorrecte des registres après un appel au runtime.

5.4.3 Lien entre comportement observé et code assembleur

Un aspect fondamental de cette méthode de débogage est la mise en relation directe entre le comportement observé à l'exécution et le code assembleur correspondant. Par exemple :

- un affichage manquant peut être expliqué par l'absence d'un appel à `__print_int` ou `__print_float`,
- un résultat incorrect peut provenir d'une valeur écrasée avant son utilisation,
- un comportement non déterministe peut révéler une dépendance implicite à un registre non initialisé.

Dans le cas du test `complex.deca`, la lecture du fichier `.ass` a permis de constater que certaines valeurs intermédiaires étaient perdues avant l'affichage final, expliquant ainsi l'absence de sortie malgré une exécution sans erreur apparente.

5.4.4 Limites de cette approche de débogage

Si la lecture du code assembleur s'est révélée extrêmement efficace pour comprendre les erreurs, elle présente néanmoins plusieurs limites :

- elle nécessite une bonne maîtrise de l'architecture ARM,
- elle est chronophage et difficilement automatisable,
- elle ne permet pas de détecter facilement toutes les erreurs de logique.

Cependant, dans le cadre de ce projet, cette approche reflète fidèlement la réalité du débogage bas niveau sur architecture réelle, où la compréhension du code machine généré devient un outil indispensable pour garantir la correction de l'exécution.

Cette méthode de débogage par inspection du code assembleur constitue ainsi un complément essentiel aux tests automatisés et explique en grande partie les corrections progressives apportées au back-end ARM du compilateur Deca.

5.5 Comportements non déterministes et erreurs de type *Bus error*

L'un des problèmes les plus significatifs rencontrés lors de la validation concerne l'apparition de résultats non déterministes et d'erreurs critiques de type :

```
qemu: uncaught target signal 7 (Bus error) - core dumped
```

Pour un même programme (`complex.deca`), les résultats observés variaient fortement selon les exécutions successives :

- affichage de valeurs correctes (par exemple 2147483647),
- affichage de 0 ou d'un résultat incohérent,
- arrêt brutal de l'exécution avec une erreur de type *Bus error*.

Ces symptômes sont caractéristiques d'erreurs de gestion mémoire ou de registres, telles que :

- écrasement de registres encore utilisés,
- mauvaise restauration de la pile après un appel de fonction,
- désalignement de la pile lors de l'exécution du runtime.

La répétition volontaire des mêmes commandes a permis de confirmer le caractère structurel de ces erreurs, et non leur origine dans les tests eux-mêmes.

5.6 Validation incrémentale et correction par commits ciblés

La validation a été étroitement couplée à une stratégie de correction incrémentale, matérialisée par des commits fréquents et ciblés. Chaque correction faisait suite à l'identification précise d'un problème lors de l'exécution ARM.

Par exemple, un problème lié à l'évaluation paresseuse des opérateurs logiques et à la gestion du court-circuit a conduit à une modification coordonnée de plusieurs classes :

```
git add And.java Or.java Divide.java
git commit -m "Correction des test de court circuit en changeant divide.java"
git push
```

De même, l'implémentation manquante de la génération ARM pour certains littéraux a été validée par des tests spécifiques, suivis d'un commit dédié :

```
git commit -m "implementation de codeGenARM pour BooleanLiteral.java"
```

Ces commits sont directement liés aux résultats observés lors de l'exécution de tests ARM et illustrent la nature itérative de la validation.

5.7 Validation du runtime ARM et des entrées/sorties

Les tests d'affichage (`print_all_types.deca`, `boolean_as_value.deca`) ont permis de valider partiellement le runtime ARM, notamment pour :

- l'affichage des entiers et des booléens,
- la gestion des chaînes de caractères,
- l'enchaînement de plusieurs appels d'affichage.

Cependant, certains tests ont révélé des sorties incomplètes ou incorrectes, mettant en évidence des limitations persistantes du runtime, en particulier lors de la combinaison de plusieurs types ou de conversions implicites.

5.8 Synthèse de la méthode de validation

La méthode de validation adoptée repose sur une confrontation systématique entre le code généré et son comportement réel à l'exécution sur une architecture ARM simulée. Les erreurs rencontrées, notamment les *Bus error* et les résultats non déterministes, ont mis en évidence les difficultés inhérentes à la gestion manuelle des registres et de la pile.

Cette validation rigoureuse, combinant scripts automatisés, tests manuels et corrections incrémentales, a permis de stabiliser progressivement le back-end ARM, tout en identifiant clairement les limitations actuelles du runtime et de la gestion mémoire. Elle constitue une base solide pour l'évaluation finale de l'implémentation, présentée dans la section suivante.

6 Validation de l'implémentation

Cette section présente une évaluation détaillée de l'extension ARM du compilateur Deca, en s'appuyant sur les résultats obtenus lors de la phase de validation. Contrairement aux sections précédentes, l'objectif n'est plus de décrire les choix de conception ou les algorithmes, mais d'analyser concrètement le comportement du compilateur face aux programmes de test, d'identifier précisément les fonctionnalités couvertes, ainsi que les limitations structurelles encore présentes.

La validation de l'implémentation met en évidence le caractère partiellement fonctionnel du back-end ARM : une part significative du langage Deca est correctement prise en charge, mais certaines constructions révèlent des failles liées à la gestion des types, des registres et du runtime.

6.1 Couverture fonctionnelle effective

Une première catégorie de tests permet de valider que le compilateur ARM est capable de générer et d'exécuter correctement un nombre important de programmes Deca, couvrant les fonctionnalités de base du langage.

Les tests suivants sont désormais pleinement fonctionnels :

- `ecrit.deca` : affichage simple validé,
- `boolean_as_value.deca` : gestion correcte des booléens comme valeurs,
- `circuit.deca` : court-circuit logique correctement implémenté,
- `conversion.deca` et `conversion_implicit.deca` : conversions int/float corrigées,
- `factorial.deca` : boucles et calculs arithmétiques validés,
- `group_bool.deca` : combinaisons logiques fonctionnelles,
- `mixed_arith.deca` et `mixed_arith_logic.deca` : expressions mixtes stabilisées,
- `no_operation.deca` : génération minimale correcte,
- `polynomy.deca`, `sum_square.deca`, `precedence.deca` : priorités opératoires respectées,
- `unary.deca`, `while_imbriq.deca`, `while_nonenter.deca` : structures de contrôle fonctionnelles,
- `wrap_around.deca` : correction du cas particulier -0.

Ces résultats montrent que le back-end ARM couvre désormais :

- l'évaluation des expressions arithmétiques entières et flottantes,
- les conversions explicites et implicites entre types numériques,
- les opérateurs booléens et les comparaisons simples,
- les structures de contrôle conditionnelles et itératives,
- l'affichage de valeurs simples via le runtime ARM.

D'un point de vue fonctionnel, le compilateur ARM est donc capable d'exécuter des programmes Deca non triviaux, incluant des calculs complexes et des structures imbriquées.

6.2 Cas partiellement couverts et comportements incorrects

Malgré cette couverture significative, plusieurs tests révèlent des comportements incorrects ou incomplets, sans provoquer nécessairement une erreur fatale à l'exécution.

Par exemple :

- `complex.deca` : le programme s'exécute sans erreur, mais n'affiche aucun résultat,
- `exhaustive_comparison.deca` : seules les comparaisons évaluées à faux sont affichées,
- `hexa.deca` : les résultats des sommes ne sont pas affichés correctement,
- `print_sideeff.deca` : les effets de bord sont mal pris en compte,
- `print_all_types.deca` : certaines combinaisons de types ne sont pas correctement gérées.

Ces erreurs ne relèvent pas d'un problème syntaxique ou contextuel, mais d'une génération de code ARM incomplète ou incorrecte. Elles sont généralement liées à :

- une mauvaise propagation des valeurs de retour dans les registres,
- une confusion entre registres entiers (`rX`) et registres flottants (`sX`),
- un enchaînement incorrect des appels au runtime lors de plusieurs affichages successifs.

Ces cas illustrent les limites actuelles de la gestion fine des types et de la discipline des registres dans le back-end ARM.

6.3 Erreurs critiques et instabilités d'exécution

Une troisième catégorie de tests met en évidence des erreurs plus graves, se traduisant par des comportements non déterministes ou des fautes d'exécution lors de l'exécution sous QEMU.

Les tests suivants sont particulièrement révélateurs :

- `decl_init.deca` : crash lors de la somme de flottants,
- `reg_priority_test.deca` : arrêt brutal avec *core dump*.

Ces erreurs se manifestent typiquement par le message :

```
qemu: uncaught target signal 7 (Bus error) - core dumped
```

De tels comportements sont caractéristiques de problèmes bas niveau, notamment :

- désalignement de la pile,
- écrasement de registres encore utilisés,
- non-respect des conventions d'appel lors des appels au runtime,
- mélange incohérent entre registres flottants et entiers.

Le caractère parfois non reproductible de ces erreurs (résultats différents entre deux exécutions successives d'un même programme) confirme leur origine structurelle dans la génération de code, plutôt que dans les tests eux-mêmes.

6.4 Limites structurelles du compilateur ARM

Au-delà des tests individuels, l'analyse globale de l'implémentation permet d'identifier plusieurs limites structurelles du back-end ARM.

Tout d'abord, la gestion des registres repose sur une stratégie simple et locale, sans allocation globale ni analyse de durée de vie. Cette approche, bien que adaptée à un cadre pédagogique, atteint rapidement ses limites lorsque les expressions deviennent complexes ou fortement imbriquées.

Ensuite, le runtime ARM reste minimal et ne couvre qu'un sous-ensemble des besoins du langage Deca. En particulier :

- certaines erreurs à l'exécution sont signalées mais ne provoquent pas d'arrêt conforme aux attentes,
- la gestion des flottants repose sur des conventions fragiles,
- les conversions successives peuvent introduire des incohérences.

Enfin, certaines constructions du langage Deca ne sont pas entièrement couvertes par l'extension ARM, notamment :

- les priorités complexes de registres,
- certaines interactions entre effets de bord et affichage,
- les cas limites impliquant simultanément booléens, flottants et entiers.

6.5 Bilan global de l'implémentation

Malgré ces limitations, l'extension ARM atteint un objectif essentiel : démontrer qu'un compilateur pédagogique initialement conçu pour une machine abstraite peut être étendu pour cibler une architecture réelle, au prix d'une complexité accrue et de contraintes strictes.

Le back-end ARM couvre aujourd'hui :

- une large partie des constructions du langage Deca,
- des expressions arithmétiques et logiques complexes,
- des structures de contrôle réalistes,
- un runtime fonctionnel pour les entrées/sorties simples.

Les failles observées ne remettent pas en cause la cohérence globale de l'extension, mais soulignent la difficulté inhérente à la génération de code bas niveau et l'importance d'une gestion rigoureuse des registres et de la pile.

Cette validation met ainsi en évidence à la fois la solidité des choix effectués et les axes d'amélioration nécessaires pour transformer cette extension pédagogique en un back-end ARM pleinement robuste.